# SIMATS ENGINEERING

## MODEL PRACTICAL EXAMINATION – Aug._2026

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1709**

| **Register Number** | **DHANUSH.G** |
|---|---|
| **Name** | **192512132** |

**Set 1**

1. Write a python program for 8-puzzle problem consists of a 3x3 grid containing 8 numbered tiles and one empty space. The objective is to move the tiles to achieve the goal state by sliding them into the empty space.
   Initial State:
   1 2 3
   4 6
   7 5 8
   Goal State:
   1 2 3
   4 5 6
   7 8
2. Write a python program for the following CRYPT arithmetic (or alphameric) puzzles involve replacing letters with digits to form valid arithmetic equations. Each letter represents a unique digit, and no number starts with a zero. For example:
   SEND
   + MORE
   ---
   MONEY
   Where each letter represents a digit, and the sum must be valid. The same letter always represents the same digit.
3. Write a prolog program for monkey banana problem.
4. Write a Prolog program to find items and its location using following facts
   location(Record, Table).
   location(Spoon, kitchen).
   location(TV, Hall).

| **Laboratory Exam Component - 100 marks** | | | |
|---|---|---|---|
| **Programs (4)** | | | |
| **Description** | **Marks Distribution** | **Total Marks** | **Marks Obtained** |
| Aim & Algorithm | 2.5*4 = 10 Marks | 10 | |
| Programs | 10*4 = 30 Marks | 40 | |
| Debugging | 2.5*4 = 10 Marks | 10 | |
| Input & Output | 05*4 = 20 Marks | 20 | |
| Result | 2.5*4 = 10 Marks | 10 | |
| GitHub content | 10 Marks | 10 | |
| **Total Lab Exam Marks** | | **100** | |

## EXPERIMENT 1 – 8-PUZZLE PROBLEM USING PYTHON

### AIM:-

To write and execute a Python program to solve the **8-Puzzle problem** using **Breadth First Search (BFS)** and obtain the sequence of moves from the initial state to the goal state.

### ALGORITHM:-

1. Start the program.
2. Represent the empty space by 0.
3. Define the initial state and goal state.
4. Store the initial state in a queue.
5. Maintain a set of visited states.
6. Remove the first state from the queue.
7. Check whether the current state is the goal state.
8. If the goal is reached, display the solution path.
9. Otherwise, find the position of the empty space.
10. Generate possible movements:
    - Up
    - Down
    - Left
    - Right
11. Add unvisited states to the queue.
12. Repeat until the goal state is reached.
13. Display the number of moves and all intermediate states.
14. Stop.

### PROGRAM:-

```
from collections import deque

def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i+1], state[i+2])
    print()


def get_neighbors(state):
    neighbors = []

    zero = state.index(0)
    row = zero // 3
    col = zero % 3

    moves = [
        (-1, 0),   # Up
        (1, 0),    # Down
        (0, -1),   # Left
        (0, 1)     # Right
    ]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
```

```
            new_zero = new_row * 3 + new_col

            new_state = list(state)

            new_state[zero], new_state[new_zero] = \
               new_state[new_zero], new_state[zero]

            neighbors.append(tuple(new_state))

    return neighbors


def solve_puzzle(initial, goal):

    queue = deque()
    queue.append((initial, [initial]))

    visited = set()
    visited.add(initial)

    while queue:

        current, path = queue.popleft()

        if current == goal:
            return path

        for neighbor in get_neighbors(current):

            if neighbor not in visited:

                visited.add(neighbor)

                queue.append(
                   (neighbor, path + [neighbor])
                )

    return None


# 0 represents empty space

initial = (
    1, 2, 3,
    4, 6, 0,
    7, 5, 8
)

goal = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)


solution = solve_puzzle(initial, goal)


if solution:

    print("Solution Found!")
    print("Number of moves:", len(solution) - 1)
```

```
    print()

    for step, state in enumerate(solution):

        print("Step", step)
        print_state(state)

else:

    print("No solution exists.")
```

**DEBUGGING:-**

**Common Error 1**
NameError: name 'deque' is not defined
**Solution:**
Add:
from collections import deque
**Common Error 2**
ValueError: tuple.index(x): x not found
**Cause:** Empty space 0 is missing.
**Solution:** Make sure the puzzle contains exactly one 0.
**Common Error 3**
No solution exists.
**Solution:** Check whether the initial puzzle configuration is solvable.

**INPUT & OUTPUT:-**

```
Solution Found!
Number of moves: 3

Step 0
1 2 3
4 6 0
7 5 8

Step 1
1 2 3
4 0 6
7 5 8

Step 2
1 2 3
4 5 6
7 0 8

Step 3
1 2 3
4 5 6
7 8 0
```

**RESULT:-**

Thus, the Python program for solving the 8-Puzzle problem using BFS was successfully executed, and the initial state was transformed into the goal state.

## EXPERIMENT 2:- CRYPT-ARITHMETIC PUZZLE

**AIM:-**

To write and execute a Python program to solve the crypt-arithmetic puzzle:

```
  SEND
+ MORE
------
 MONEY
```

by assigning a unique digit to each letter.

**ALGORITHM:-**

1. Start the program.
2. Identify all unique letters.
3. Generate possible digit combinations from 0 to 9.
4. Assign a unique digit to every letter.
5. Ensure that S and M are not zero.
6. Convert SEND, MORE and MONEY into numerical values.
7. Check whether:

SEND + MORE = MONEY

8. If the equation is valid, display the solution.
9. Otherwise, try the next combination.
10. Stop after finding a valid solution.

**PROGRAM:-**

```
from itertools import permutations

letters = "SENDMORY"

for digits in permutations(range(10), 8):

    S, E, N, D, M, O, R, Y = digits

    # S and M cannot be zero
    if S == 0 or M == 0:
        continue

    SEND = 1000*S + 100*E + 10*N + D

    MORE = 1000*M + 100*O + 10*R + E

    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y

    if SEND + MORE == MONEY:

        print("Solution Found!")

        print("S =", S)
        print("E =", E)
        print("N =", N)
```

```
        print("D =", D)
        print("M =", M)
        print("O =", O)
        print("R =", R)
        print("Y =", Y)

        print()
        print("SEND  =", SEND)
        print("MORE  =", MORE)
        print("MONEY =", MONEY)

        print()
        print(SEND, "+", MORE, "=", MONEY)

        break
```

**DEBUGGING:-**

**Error 1**
NameError: name 'permutations' is not defined
**Solution:**
Add:
from itertools import permutations
**Error 2**
Repeated digits are assigned to different letters.
**Solution:**
Use:
permutations(range(10), len(letters))
This automatically provides unique digits.
**Error 3**
A number begins with zero.
**Solution:**
Use:

```
if mapping['S'] == 0 or mapping['M'] == 0:
    continue
```

**INPUT & OUTPUT:-**

```
Solution Found!
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2

SEND  = 9567
MORE  = 1085
MONEY = 10652

9567 + 1085 = 10652
```

**RESULT:-**

Thus, the Python program successfully solved the **SEND + MORE = MONEY crypt-arithmetic puzzle** using permutation and constraint checking.

# EXPERIMENT 3 – MONKEY BANANA PROBLEM USING PROLOG

**AIM:-**

To write and execute a Prolog program to solve the **Monkey Banana problem** using facts, rules and logical state transitions.

**ALGORITHM:-**

1. Start the program.
2. Define the initial state.
3. Represent the monkey position.
4. Represent the box position.
5. Represent the banana position.
6. Move the monkey towards the box.
7. Push the box below the bananas.
8. Climb onto the box.
9. Grab the bananas.
10. Display the sequence of actions.
11. Stop.

**PROGRAM:-**

```
% Monkey Banana Problem

initial_state(
    state(at(door), at(window), at(ceiling), onfloor, hasnot)
).

move_monkey(
    state(_, Box, Banana, Floor, Has),
    state(Box, Box, Banana, Floor, Has)
).

push_box(
    state(Monkey, _, Banana, Floor, Has),
    state(Monkey, Banana, Banana, Floor, Has)
).

climb_box(
    state(Monkey, Box, Banana, _, Has),
    state(Monkey, Box, Banana, onbox, Has)
).

grab_banana(
    state(Monkey, Box, Banana, onbox, hasnot),
    state(Monkey, Box, Banana, onbox, has)
).

solve(Actions) :-
    initial_state(S0),
    move_monkey(S0, S1),
```

```
    push_box(S1, S2),
    climb_box(S2, S3),
    grab_banana(S3, _),
    Actions = [
        move_monkey,
        push_box,
        climb_box,
        grab_bananas
    ].
```

**DEBUGGING:-**

**Error 1**
ERROR: Undefined procedure
**Solution:**
Load the file correctly:
?- [monkey].
**Error 2**
If the query gives no result, check that predicate names in the query exactly match the program.
Use:
?- solve(Actions).
**Error 3**
Incorrect capitalization.
In Prolog:
Monkey
is a variable, while:
monkey
is an atom.
Therefore, use lowercase names for fixed values.

**INPUT & OUTPUT:-**

```
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/LENOVO/Desktop/Monkey Banana Problem.pl').
compiling C:/Users/LENOVO/Desktop/Monkey Banana Problem.pl for byte code...
C:/Users/LENOVO/Desktop/Monkey Banana Problem.pl compiled, 37 lines read - 3289 bytes written, 11 ms

yes
| ?- solve(Actions).

Actions = [move_monkey,push_box,climb_box,grab_bananas]

yes
| ?-
```

**RESULT:-**

Thus, the Prolog program successfully solved the **Monkey Banana problem** using logical rules and state transitions.

## EXPERIMENT 4: ITEM LOCATION USING PROLOG

**AIM:-**

To write and execute a Prolog program to find items and their locations using facts and logical queries.

**ALGORITHM:-**

1. Start SWI-Prolog.
2. Define the location facts.
3. Create a rule to find the location of an item.
4. Give an item as a query.
5. Prolog searches the knowledge base.
6. Display the location of the item.
7. Repeat for other items if required.
8. Stop.

**PROGRAM:-**

```
% Location facts

location(record, table).
location(spoon, kitchen).
location(tv, hall).

% Rule

find_location(Item, Place) :-
    location(Item, Place).
```

**DEBUGGING:-**

**Error 1**
Incorrect capitalization:
location(Record, Table).
Here Record and Table are variables.
Use:
location(record, table).
**Error 2**
If the program is not loaded:
?- [location].
**Error 3**
Incorrect query.
Correct:
?- find_location(spoon, Place).

**INPUT & OUTPUT:-**

```
| ?- consult('C:/Users/LENOVO/Desktop/Item Location Program.pl').
compiling C:/Users/LENOVO/Desktop/Item Location Program.pl for byte code...
C:/Users/LENOVO/Desktop/Item Location Program.pl compiled, 7 lines read - 637 bytes written, 4 ms

yes
| ?- find_location(record, Place).

Place = table

(15 ms) yes
| ?- find_location(tv, Place).

Place = hall

yes
| ?-
```

**RESULT:-**

Thus, the Prolog program was successfully executed to **find the items and their corresponding locations** using facts and logical queries.